

A Co-Optimization of Software and Hardware for PCIe-Based Small Packet DMA Transfer

Xiaotian Gu^{1b}, Lisong Shao, Ningfeng Bai^{1b}, *Member, IEEE*, Guosheng Zhang, and Xinyi Zhang

Abstract—To meet the development needs of high-performance networks, the efficiency of PCIe DMA with small packet transmissions emerges as a critical performance bottleneck. For the purpose improving DMA transfer for small packets over PCIe, we focus on the symbiotic co-optimization of software and hardware. By enhancing the software driver with descriptor prefetching, harnessing the outstanding capability at the hardware level, and using multicore for parallel processing, the system's bandwidth has been significantly improved, particularly for the small packets transfers. Postoptimization, we test the DMA read and write bandwidth and utilization rate performance on VC709 FPGA. In a randomized test scenario, the average DMA read bandwidth is approximately 5.3 GB/s, representing a 123% improvement compared to the unoptimized system bandwidth. The average DMA write bandwidth is approximately 5.8 GB/s, representing a 136% improvement compared to the unoptimized system bandwidth. Additionally, in the randomized test scenario, the average read and write latencies improved by 15.12% and 23.96%, respectively.

Index Terms—Descriptors prefetch, DMA, FPGA, multicore parallelism, outstanding capability, PCIe.

I. INTRODUCTION

WITH the development of high-performance networks, the demand for high throughput, low-latency networks is increasing. According to the analysis of data center traffic characteristics by the Benson et al. [1], 60% of data flows do not exceed 1 KB in size, and these small data flows are sensitive to latency. Therefore, it is necessary to optimize the bandwidth and latency of small packet transmission in network communication.

Although PCIe DAM is essential for achieving high-speed data transfers, its efficiency with small packet transmissions continues to challenge the overall system performance. Renowned frameworks RIFFA 2.0 [2], exhibits robust stability in handling large packet data but demonstrates bad bandwidth and latency performance with the small packet data. The work of Choi et al.'s [3] and Kasai and Osana [4], have also encountered issues with low bandwidth utilization when transferring 256B small data packets, with PCIe bandwidth utilization even falling below 20%, and this study did not optimize for latency.

Manuscript received 20 May 2024; revised 16 June 2024; accepted 14 July 2024. Date of publication 17 July 2024; date of current version 13 February 2025. This manuscript was recommended for publication by A. Shrivastava. (*Corresponding author: Ningfeng Bai.*)

Xiaotian Gu and Ningfeng Bai are with the School of Electronic Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: bnfeng@seu.edu.cn).

Lisong Shao is with Phytium Technology Company Ltd., Changsha, China.

Guosheng Zhang is with the School of Software, Southeast University, Nanjing 215000, China.

Xinyi Zhang is with the School of Integrated Circuit, Southeast University, Nanjing 215000, China.

Digital Object Identifier 10.1109/LES.2024.3429544

Tanwar et al.'s [5] improved the bandwidth for 128B packet transmission to 0.4625 GB/s. The work of Zhao et al.'s [6] has optimized the bandwidth for transmitting 128B packets to approximately 3.4 GB/s. However, the data transmission latency has reached the ms level.

In response to the bandwidth and latency challenges associated with small packet transfers over PCIe, we propose a collaborative software and hardware optimization solution on Xilinx VC709 FPGA based on Xilinx XDMA [7], [8], [9], [10], [11], [12].

- 1) *Descriptor Prefetch*: On the software side, we have enhanced the descriptor prefetching Linux driver [13] for scatter gather DMA, mitigating the time overhead caused by continuous hardware requests for descriptors in the process of transferring small packets to the Host memory.
- 2) *Outstanding Capability*: Simultaneously, at the hardware level, we have leveraged PCIe DMA configuration space register [14] models to enhance outstanding capabilities.
- 3) *Multicore Parallelism*: Furthermore, by employing a multicore parallel approach to address the interval between initiation of transfer tasks on the Host side, we have significantly improved the efficiency of the data transfers.

II. BACKGROUND

The Host transfers data with FPGA via PCIe link. The driver creates a scatter gather list in the Host memory, and initiates the DMA read/write operations by writing to the control registers in the DMA controller using PIO. The DMA channel then requests operations on the descriptors and reads the descriptor linked list from the Host memory. Subsequently, the read/write operations are executed based on the information provided by the scatter gather list.

SG DMA can link scattered memory spaces together through a linked list and provides higher flexibility in DMA transmission through the use of descriptors. This method is well suited for scenarios involving large-scale data transfers.

A significant portion of the overall transfer process is spent on requesting descriptors from the Host memory, reading descriptors into the descriptor FIFO in the DMA Channel, which affects the efficiency of the transfer.

Therefore, optimizing the descriptor queue strategy in both the driver and the hardware is required to achieve excellent efficiency in small packet transmission.

III. ARCHITECTURE AND OPTIMIZATION

A. Overall Design

The system includes a PCIe card, a Linux kernel-based driver, and a test application. The hardware architecture is

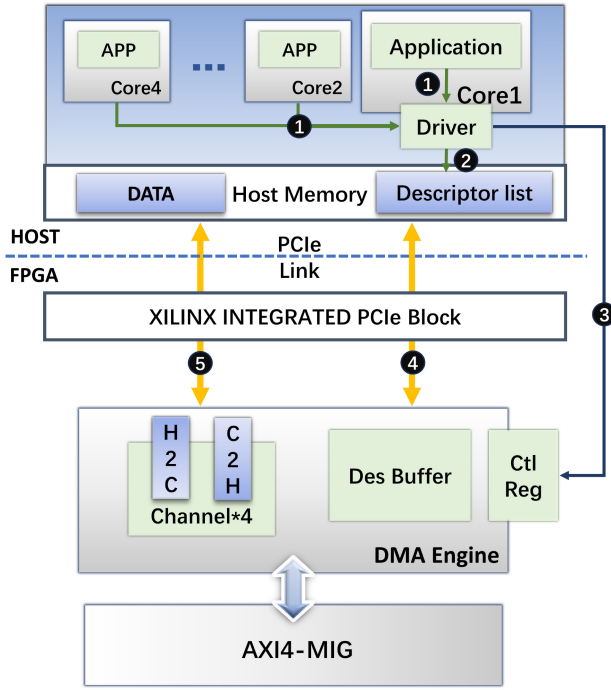


Fig. 1. Overall system design.

founded upon the VC709 development board. It is important to note that we have configured four channels for XDMA in the hardware system. In parallel, a Linux kernel driver and test application has been engineered to evaluate the data transfer rates between the Host and the FPGA board. Based on the system structure designed by the related work [8], [9], [10], [11], [12], our software-hardware co-design is shown in Fig. 1.

This section represents the entry point for the entire communication between the FPGA and the Host. Communication between the Host and the FPGA is conducted through a PCIe link. Initially, a Xilinx PCIe hard IP converts the PCIe protocol into the AXI-stream data transmission. AXI-stream is a protocol within the AXI [15] protocol suite that is suitable for address free high-volume data transmission. Subsequently, the data is transmitted through an XDMA [16] to the MIG [17] interface via the AXI4 bus. MIG stands for the memory interface generator and serves as the interface for the DDR3 memory.

To improve the bandwidth utilization rate of the PCIe link, we optimize the system through the software and hardware method.

B. Design Challenge and Solutions

In the initial design of this work, the clock differences between the AXI interface in PCIe DMA and the DDR MIG AXI interface were overlooked, and no cross-clock domain handling was implemented. This oversight resulted in continuous data transfer errors.

In subsequent designs, an MMCM module is added to generate a 200-MHz clock for DDR MIG using the 250-MHz clock source from PCIe. Additionally, an AXI crossbar is designed to handle the cross-clock domain data transfer. Internally, an asynchronous FIFO is used within the AXI bus, with the asynchronous FIFO's clock domains being the PCIe DMA AXI interface and the DDR MIG AXI interface. This

design synchronizes data transfer between the PCIe DMA AXI side's 250-MHz clock domain and the DDR MIG AXI side's 200-MHz clock domain. After implementing these error handling measures, the data transfer errors no longer occurred.

C. Improvement Implementation

1) *Descriptor Prefetch*: We add descriptor prefetching process in the PCIe Linux driver. It is necessary to iterate through all the DMA descriptors and invoke the `xdma_get_next_adj` and `xdma_desc_adjacent` functions to compute the address of the next descriptor for each descriptor. The computed address is then written into the address field of the current descriptor. These descriptors are organized into a scatter gather linked list data structure and transmitted collectively to the XDMA descriptor buffer. This enables the XDMA to directly process the next descriptor based on the address, thereby reducing the time loss associated with repeatedly requesting individual descriptors from the DMA. In this system, the maximum number of descriptors that can be prefetched is 64.

2) *Outstanding Capability*: In PCIe transactions, for write transactions that do not require a response, using descriptor prefetching can improve the actual PCIe bandwidth of the system because the data payload is contained within the DMA write transaction layer packet (TLP). However, for the DMA read transactions, descriptor prefetching is only applicable to the read request, while the actual data payload is contained within the read response TLP.

To address the above issue, optimization for read response transactions is necessary. In this regard, the extended tag field capability [14] of the configuration space in the hardware is enabled. The extended tag field capability is used for the bit width of the tag field in the PCIe TLP header. When this capability is enabled, the tag field in the PCIe TLPs is expanded from 5 to 8 bits. Once enabled, it can accommodate up to 64 outstanding transactions.

3) *Multicore Parallel Process*: Based on the XDMA's configuration capability, four channels are enabled, allowing four CPU cores to perform parallel tasks on the four channel, respectively. Each core corresponds to one channel.

Parallel processing can reduce the waiting time for the individual tasks; makes the most of four channels, allowing for more efficient utilization of CPU resources in a multicore environment. By employing this approach to leverage the multichannel capability of the XDMA, the system improves the resource utilization and the system throughput while handling small packet transfers.

IV. TEST AND EVALUATION OF THE SYSTEM

A. Evaluation Model and Experiment Setup

The evaluation of bandwidth and latency is divided into single data volume tests and randomized tests that simulate the data center workloads.

The model for evaluating the bandwidth with a single data volume test is as follows:

$$BW_{\text{experi}} = \frac{\text{Data}_{\text{size}} \times \text{iterations} \times f}{1000 \times \text{Clock}_{\text{cyclecount}}}. \quad (1)$$

The $\text{Data}_{\text{size}}$ represents the data size in one transfer. The iterations is the number of repetitions. The f is the frequency of AXI bus, which is 250 MHz. The $\text{Clock}_{\text{cyclecount}}$ stands for the total number of transmission clock cycles.

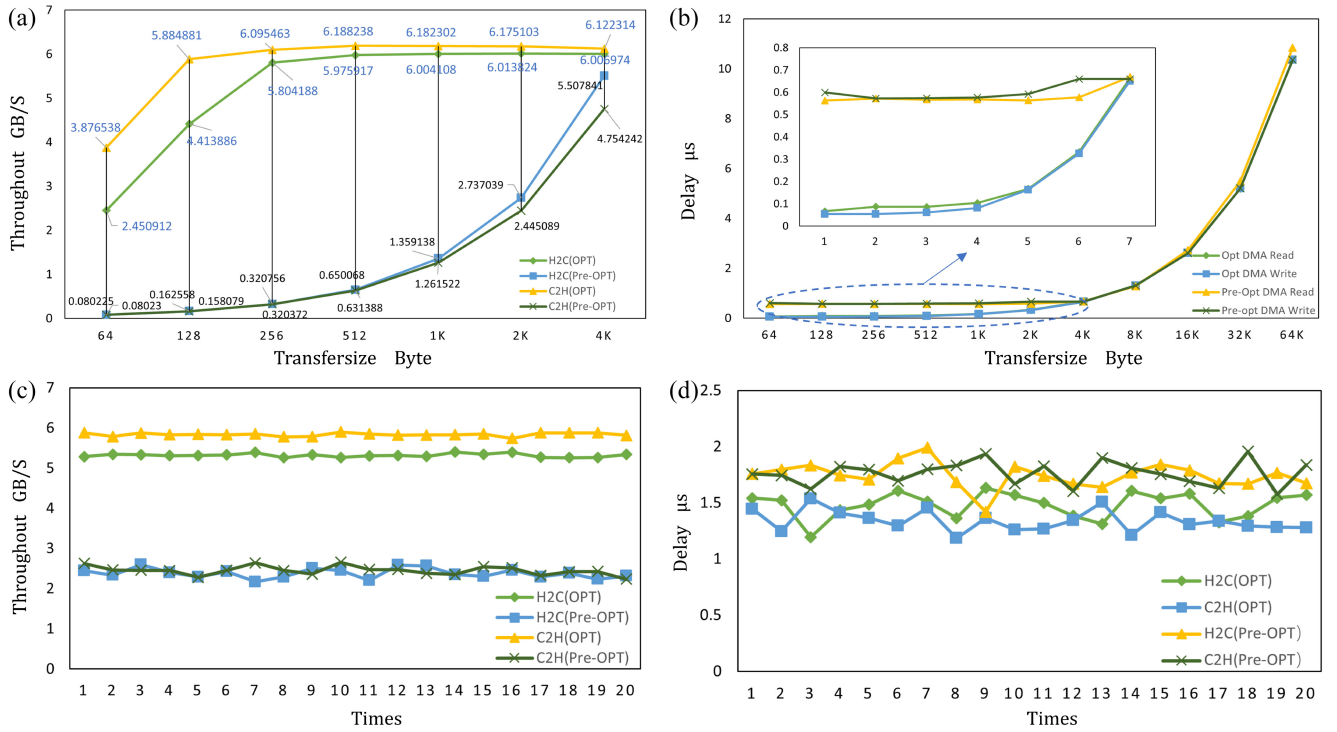


Fig. 2. (a) Comparison of DMA read/write bandwidth in single data size scenarios. (b) Comparison of DMA read/write latency in single data size scenarios. (c) Comparison of DMA read/write average bandwidth in randomized test scenarios. (d) Comparison of DMA read/write average latency in randomized test scenarios.

TABLE I
EVALUATION MODEL PARAMETER

Parameter	Value
MPS(Max payload size)	256B
MRRS(Max Read Request size)	128B
Mrd _{Overhead}	28B
Cpld _{Overhead}	28B
Mwr _{Overhead}	28B
LW(Link Width)	8
ES(Encoding Scheme)	128b/130b
trans _{num}	500
test _{num}	20

TABLE II
TEST PLATFORM PARAMETER

Test Platform	Intel
Host CPU	XEON E5-2630 v3 @ 2.40GHz
Host Architecture	X86_64
Host memory	Micron 64GB DDR4 1866MHz
FPGA Fabric	Xilinx Virtex 7 VC709 FPGA
PCIe	Gen3 x8, 8GB/s
Device memory	Micron 4GB DDR3 1600MHz
Operating System	Linux ubuntu 5.4.0

The average throughput and latency evaluation model in randomized tests is as follows:

$$BW_{average} = \frac{\sum_{j=1}^{test_{num}} \sum_{i=1}^{trans_{num}} BW_i}{test_{num} \times trans_{num}} \quad (2)$$

$$Latency_{average} = \frac{\sum_{j=1}^{test_{num}} \sum_{i=1}^{trans_{num}} latency_i}{test_{num} \times trans_{num}} \quad (3)$$

The evaluation parameter in our test is listed in Table I.

B. Experiments Results and Analysis

We compare the optimized four-channel system with the nonoptimized single-channel system. In the experiments H2C refers to the data transfer from the Host to the FPGA (card) side, which is the DMA read process for the DMA engine. C2H refers to the data transfer from the FPGA (card)

side to the Host, which is the DMA write process for the DMA engine. The experiment setup configuration is listed in Table II.

1) *Analysis of the Throughput*: It can be seen from Fig. 2(a), for single-volume data tests, it is evident that the system throughput significantly increases when handling the data volumes smaller than 4 KB. The optimized system reaches a peak bandwidth of nearly 6 GB/s during transfers of approximately 256 bytes of data. In Fig. 2(c) we simulated data characteristics in a data center, testing the average bandwidth with 20 rounds of 500 random data packets. In the previous research [1], 60% of network data packets are no larger than 1 KB. Based on this, we conducted a randomized test, the optimized bandwidth shows an improvement of over 120%. The optimized read and write average bandwidths reached approximately 5.3 and 5.8 GB/s, respectively, which is basically consistent with the theory. In the unoptimized

TABLE III
RESOURCE CONSUMPTION COMPARISON

Resource	Pre-opt Utilization	Opt Utilization
LUT	61081	63769
FF	55227	58349
BRAM	69	69
Power(W)	4.695	4.672

system, small packet transfer performance is very low, so this improvement is quite reasonable.

Multicore invocation reduces the time overhead in creating and sending descriptors, enhancing the duty cycle of data periods. Descriptor prefetching and enhancements on the hardware side, such as increasing the extended tag field, essentially enhance the outstanding capability, allowing more descriptors to be sent during transmission to reduce descriptor handling overhead and improve the data period's duty ratio. On the hardware side, the capability to handle multiple tasks simultaneously reduces waiting times for the tasks. These optimization strategies are significantly beneficial for the bandwidth improvement.

2) *Analysis of the Delay*: It can be seen from Fig. 2(b) and (d), in single-volume data tests, when the data volume is less than 4 KB, there is a significant improvement in transmission latency. Randomized testing using the data distribution in simulated data centers quantifies the average latency during small volume data transfers. After optimization, the average read and write latencies are 1.48 and 1.34 μ s, respectively, representing improvements of 15.72% and 23.56% compared to the unoptimized system.

By prefetching 64 descriptors, the repeated time overhead of requesting descriptors from the Host memory during small packet transfers is reduced. Furthermore, as the operations of the DMA data transfer and descriptor prefetching are concurrent, this significantly optimizes high latency situations in small packet transfers. Additionally, multicore invocation also reduces the time overhead associated with waiting to create and send descriptors.

3) *Analysis of the Resource and Power Consumption of Design*: From the Table III, there is an increase of approximately 2688 LUTs, and about 3122 triggers compared to the unoptimized system. We increase the size of the descriptor buffer in the DMA engine to enhance its outstanding capability. Inevitably, this leads to some hardware overhead, but this overhead is relatively small, making it relatively friendly for deployment on the resource-constrained embedded devices.

4) *Comparison With Related Work*: It can be observed from Table IV that, compared to the previous similar studies, our optimized system demonstrates significant advantages in the bandwidth and latency during simulated real-data-center testing.

V. CONCLUSION AND FUTURE WORK

Our work high light the significance of optimizing the PCIe DMA data transfer processes for various packet sizes in high-speed computing environments. By enhancing the software driver with descriptor prefetching, harnessing the outstanding capability at the hardware level, and use the multichannel capability of the XDMA by using the multicore parallel

TABLE IV
COMPARISON WITH RELATED WORK

Work	PCIe	Average Bandwidth	Average Latency
Ours	Gen3 x8	5.4GB/s	1.4 μ s
[2]	Gen3 x8	<5GB/s	>2 μ s
[6]	Gen2 x8	<4GB/s	>10 μ s

process, the system's average bandwidth and average latency can both be significantly optimized.

Our future work will focus on the PCIe interface Ethernet cards. Based on the above optimization measures for small data stream transmission, we aim to develop a PCIe network card device that can adapt to optimized bandwidth and latency for small data streams. Additionally, we will use the presented UDP/IP and RoCEv2 stack for Ethernet transmission to accommodate the requirements of lossless transmission and remote direct memory access (RDMA).

REFERENCES

- [1] T. Benson, A. Akella, and D. A. Maltz, "Network traffic characteristics of data centers in the wild," in *Proc. 10th ACM SIGCOMM Conf. Internet Meas.*, 2010, pp. 267–280.
- [2] M. Jacobsen, D. Richmond, M. Hogains, and R. Kastner, "RIFFA 2.1: A reusable integration framework for FPGA accelerators," *ACM Trans. Reconfig. Technol. Syst.*, vol. 8, no. 4, p. 22, Sep. 2015.
- [3] Y. Choi, J. Cong, Z. Fang, Y. Hao, G. Reinman, and P. Wei, "A quantitative analysis on microarchitectures of modern CPU-FPGA platforms," in *Proc. 53rd ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, 2016, pp. 1–6.
- [4] S. Kasai and Y. Osana, "A driver-based approach for DMA transfer between FPGA-GPU," in *Proc. 10th Int. Symp. Comput. Netw. Workshops*, 2022, pp. 103–108.
- [5] P. K. Tanwar et al., "Zynq SoC based high speed data transfer using PCIe: A device driver based approach," in *Proc. 14th IEEE India Council Int. Conf. (INDICON)*, Roorkee, India, 2017, pp. 1–6.
- [6] Y. Zhao, X. Liu, and J. Yang, "A resource and timing optimized PCIe DMA architecture using FPGA internal data buffer," *IEICE Electron. Expr.*, vol. 16, no. 1, 2019, Art. no. 20180858.
- [7] (Xilinx, San Jose, CA USA). *VC709 Evaluation Board for the Virtex-7 FPGA User Guide*. (2013). [Online]. Available: www.xilinx.com
- [8] L. Du, Z. Zhang, J. Tong, C. Wang, and W. Wang, "Research on XDMA high-speed data transmission architecture based on PCIe," in *Proc. IEEE 6th Int. Conf. Comput. Commun. (ICCC)*, 2020, pp. 1783–1787.
- [9] L. Rota, M. Caselle, S. Chilingaryan, A. Kopmann, and M. Weber, "A PCIe DMA architecture for multi-gigabyte per second data transmission," *IEEE Trans. Nucl. Sci.*, vol. 62, no. 3, pp. 972–976, Jun. 2015.
- [10] H. Kaviani-pour, S. Muschter, and C. Bohm, "High performance FPGA-based DMA interface for PCIe," *IEEE Trans. Nucl. Sci.*, vol. 61, no. 2, pp. 745–749, Apr. 2014.
- [11] X. Q. Chen, L. Y. Zhang, and C. X. Li, "FPGA-based train onboard PCIe board design and implementation," in *Proc. 3rd Int. Conf. Electr. Eng. Mechatron. Technol. (ICEEMT)*, 2023, pp. 202–206.
- [12] Y. K. Choi, J. Cong, Z. Fang, Y. Hao, G. Reinman, and P. Wei, "A quantitative analysis on microarchitectures of modern CPU-FPGA platforms," in *Proc. Design Autom. Conf.*, Jun. 2016, pp. 1–6.
- [13] J. Adamczewski-Musch, N. Kurz, and S. Linev, "MBSPEX and PEXORNET—Linux device drivers for PCIe optical receiver DAQ and control," *IEEE Trans. Nucl. Sci.*, vol. 65, no. 2, pp. 788–795, Feb. 2018.
- [14] "PCI Express® base specification revision 3.0," PCI-SIG. 2010. [Online]. Available: www.pcisig.com
- [15] (Xilinx, Inc., San Jose, CA, USA). *SmartConnect V1.0 LogiCORE IP Product Guide Vivado Design Suite*. (2024). [Online]. Available: www.xilinx.com
- [16] (Xilinx, Inc., San Jose, CA, USA). *DMA/Bridge Subsystem for PCI Express v4.1 Product Guide Vivado Design Suite PG195 (V4.1)*. (2021). [Online]. Available: www.xilinx.com
- [17] (Xilinx, Inc., San Jose, CA, USA). *UltraScale Architecture-Based FPGAs Memory IP V1.4 LogiCORE IP Product Guide Vivado Design Suite*. (2024). [Online]. Available: www.xilinx.com